VINUNIVERSITY

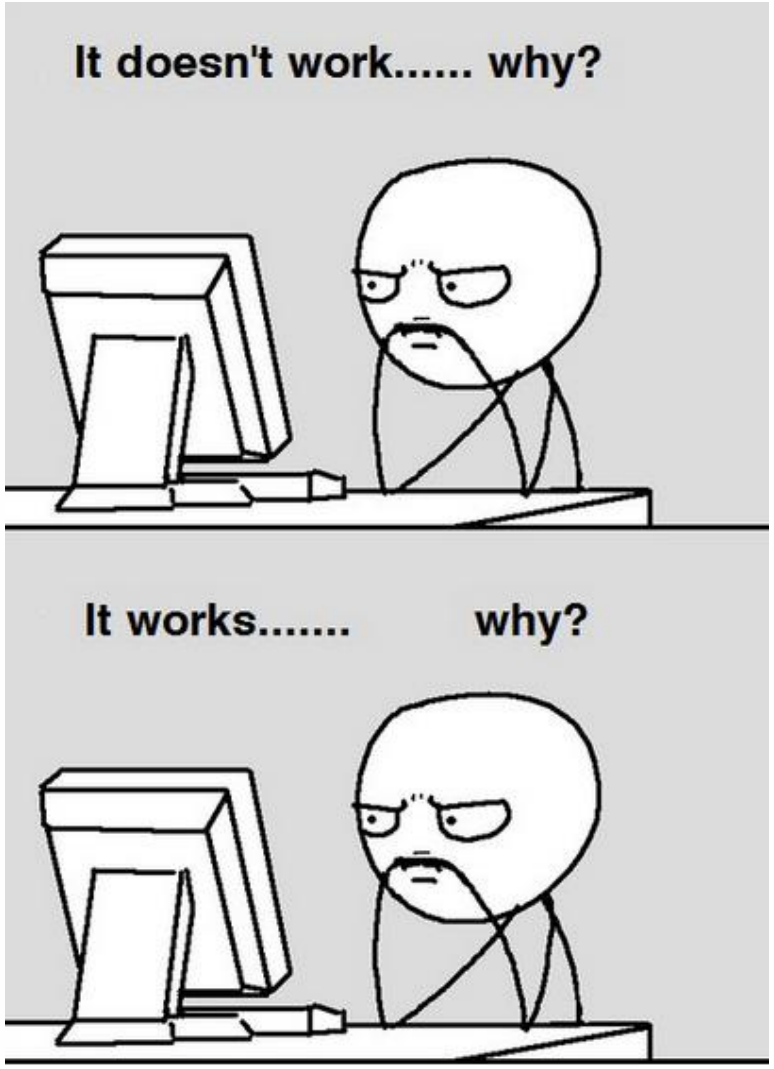
Lecture 08: Specification and Testing

Understand software specification, pre-condition, program bugs, testing and debugging

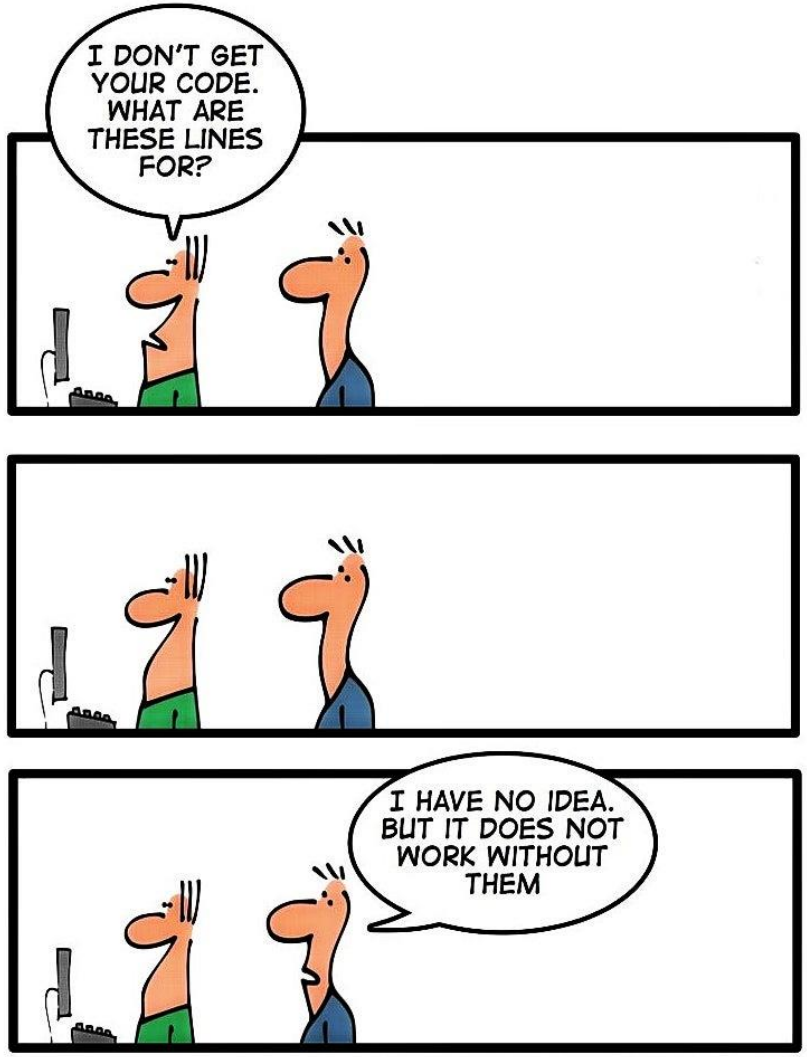
Pham Huy Hieu & Kok-Seng Wong | Fall 2025

October 8, 2025

MOTIVATIONAL EXAMPLE 1

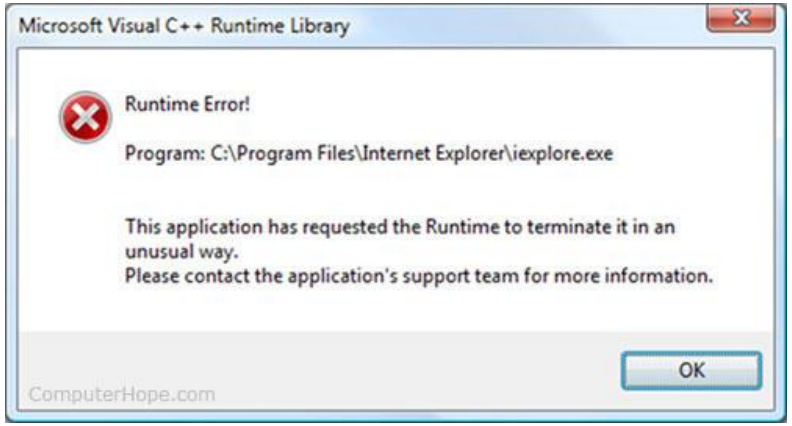


[source]



[source]

MOTIVATIONAL EXAMPLE 2





[source]



[source]

LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- understand the anatomy of a specification.
- explain the basic terminology (bug, debugging, test, testing) in programming.
- identify different types of errors in the program.
- Understand asserts and exceptions handling in the program.

ANATOMY OF A SPECIFICATION

```
def get_course_code(course_id):
    """returns the course code of Computer Science
    with 4 digits course id.The first digit represents
    course offered in a year. E.g., 3447 is a 3rd year
    course with registered id 447.

    Returns: str of form "CS-XXX"

    course_id: prefix with course year
    Precondition: course id is a 4 digit string of only numbers
    """
    return "CS" + "-" + course_id[1:]
```

short description, followed by blank line

Parameter description

Information (with format) about the return value

Precondition specifies assumptions we make about the arguments

PRECONDITION

- Precondition: a condition that must always be true just before the execution of some section of codes or before an operation in a formal specification.
- If a precondition is violated, the code section's effect becomes undefined.
- It may or may not carry out its intended work.

PRECONDITION IS A CONTRACT

- If precondition is met, the function will work!
- Else, no guarantees…

```
get_course_code("3447")
```

```
'CS-447'
```

```
get_course_code("344A")
```

```
'CS-44A'
```

Precondition violated:
No Error Message!

```
get_course_code(3447)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-34-5165cd1ac768> in <module>
----> 1 get_course_code(3447)

<ipython-input-32-28c186a0abdf> in get_course_code(course_id)
     11     Precondition: course id is a 4 digit sring of only numbers
     12     """
---> 13     return "CS" + "-" + course_id[1:]

TypeError: 'int' object is not subscriptable
```

Precondition violated:
With Error Message!

BASIC TERMINOLOGY

- Bugs: errors in a program
- Debugging: the process of finding bugs and removing them
- Testing: the process of analyzing and running a program (looking for bugs)
- Test Case: a set of input values, together with the expected output

WHAT IS DEBUGGING?

"Debugging is the process of finding, understanding, and fixing errors (bugs) in a program so that it behaves as expected."

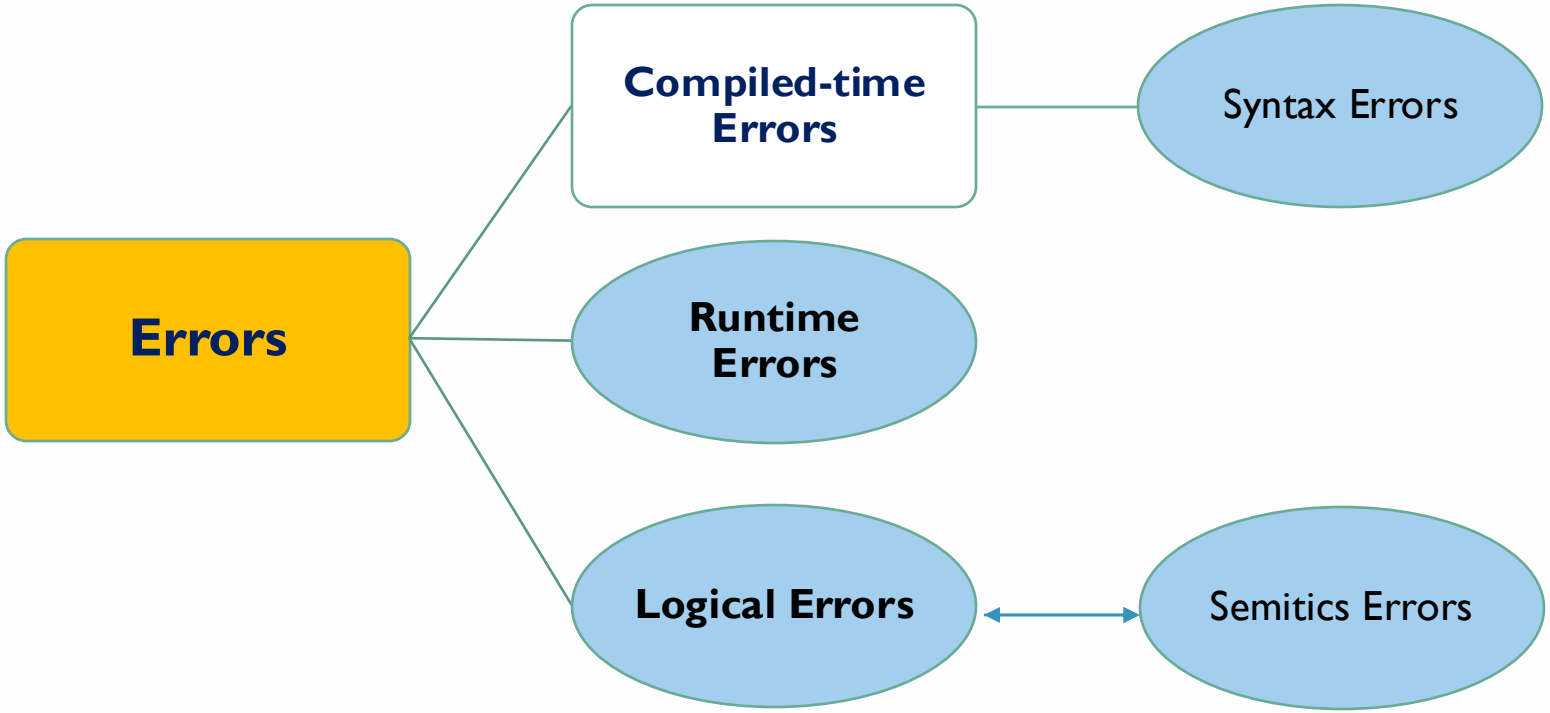
"The art of debugging is figuring out what you really told your program to do rather than what you thought you told it to do."

—Andrew Singer



[source]

ERRORS IN PROGRAM



SYNTAX ERRORS

- Python can only execute a program if the program is syntactically correct.
- Otherwise, the process fails and returns an error message.
- Syntax: structure of a program and the rules about that structure.

If there is a single syntax error anywhere in your program, Python will print an error message and quit!

Who or what typically finds syntax errors?
(a) Compiler / interpreter (b) Programmer (c) Computer

SYNTAX ERROR – EXAMPLES

```
def my_function()
    msg = "Hello World!"
    return msg
```

A

```
def my_function():
    1msg = "Hello World!"
    return 1msg
```

B

INDENTATION ERROR - EXAMPLE

- Indentation error also indicates a problem with the syntax of your program.
- More specific: it always means that there is a problem with how your code is indented.

```
def my_function():
    msg = "Hello World!"
   return msg
```

```
  File "<tokenize>", line 3
    return msg
    ^
IndentationError: unindent does not match any outer indentation level
```

An IndentationError is a kind of SyntaxError

MISUSE OF ASSIGNMENT (=) - EXAMPLES

- In Python, assignment (=) cannot be used in conditional statements.

```
if x = 100:
    print("Awesome")
```

```
x = 100:
while x = 100:
    print("Wow")
    x += 1
```

SEMANTIC ERRORS

- If there is a semantic error in your program, it will run successfully, in the sense that the computer will not generate any error messages, but it will not do the right thing.
- The problem is that the program you wrote is not the program you wanted to write.
- The meaning of the program (its semantics) is wrong.

Who or what typically finds semantic errors?
(a) Compiler / interpreter (b) Programmer (c) Computer

SEMANTIC ERROR - EXAMPLE

```
a = float(input('Enter the first number: '))
b = float(input('Enter the second number: '))

average = a+b/2
print ('The average of the two numbers you have entered is:',average)
```

RUNTIME ERRORS

- Runtime error does not appear until you run the program.
- Also known as exceptions because they usually indicate that something exceptional (and bad) has happened.
  - division by zero
  - operation on incompatible types
  - using undefined identifier
  - accessing a list element, dictionary value or object attribute which doesn't exist

Who or what typically finds runtime errors?
(a) Compiler / interpreter (b) Programmer (c) Computer

RUNTIME ERROR – EXAMPLES

- In Python, all variables are expected to be defined before use.

```
print(x)
```

A

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent
<ipython-input-16-fc17d851ef81> in <module>
----> 1 print(x)

NameError: name 'x' is not defined
```

```
for number in range(6):
    count = count + number
print("The total is:", count)
```

B

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent
<ipython-input-18-0fdcc212f8c5> in <module>
      1 for number in range(6):
----> 2     count = count + number
      3 print("The total is:", count)

NameError: name 'count' is not defined
```

Any Error?

```
Count = 0
for number in range(6):
    count = count + number
print("The total is:", count)
```

RUNTIME ERROR – ZERO_DIVISION

```
total = 0
numb1 = 3
numb2 = 4
print(numb1+numb2/total)
```

LOGICAL ERRORS

- The computer will not generate any error messages, but it will produce incorrect or undesired (unexpected) output.
- The most difficult to spot!

Who or what typically finds logical errors?
(a) Compiler / interpreter (b) Programmer (c) Computer

EXPERIMENTAL DEBUGGING

- Debugging: a popular term used to process, locating and rectifying errors in a program.
- Python's standard library contains pdb module which is a set of utilities for debugging of Python programs.
- More info: https://docs.python.org/3/library/pdb.html

PYCHARM DEBUGGING (LAB SESSION)

TEST CASES

- Test cases: Different types of inputs are used in your code to test your defined logic and produce the output.
- A test case is termed "passed" when the output from your code exactly matches the expected output.
- It helps to identify errors in the specification as well as the implementation.
- **Question**: Identify types of errors?

```
viniciusd at centos7-ny in ~/app on python?
☿ python testApp.py
F..              Unit Test Report
Start Time: 2015-07-27 21:01:54
Duration: 0:00:00.000858
Status:
    Pass: 2
    Failure: 1

Description:
Summary:
    +---------------------+------+------+------+-------+
    | Test group/Test case | Count | Pass | Fail | Error |
    +---------------------+------+------+------+-------+
    | TestStringMethods    | 3     | 2    | 1    | 0     |
    | Total                | 3     | 2    | 1    | 0     |
    +---------------------+------+------+------+-------+
    TestStringMethods
        +-------------+-----------------------------------------------------+------------+
        |  Test name  |                        Stack                        |   Status   |
        +-------------+-----------------------------------------------------+------------+
        | test_isupper | Traceback (most recent call last):                 | fail       |
        |              |   File "testApp.py", line 11, in test_isupper      |            |
        |              |     self.assertTrue('Foo'.isupper())               |            |
        |              | AssertionError: False is not true                  |            |
        | test_split   |                                                    | pass       |
        | test_upper   |                                                    | pass       |
        +-------------+-----------------------------------------------------+------------+
```

ASSERTIONS & HANDLING EXCEPTIONS

ASSERTIONS IN PYTHON

- Assertions: test your code's correctness by checking if some specific conditions remain **TRUE** during execution.
- The assertion condition is always **TRUE** when there is no bug in the program.
- If the assertion condition is **FALSE**, then the assertion raises an exception and terminates the program's execution.
- It is often used to check pre-conditions and post-conditions in the programs.

Before executing function logic

Expected outcomes after performing an operation

EXAMPLE 1

```
my_age = int(input("How old are you?"))
assert my_age > 0, f"Alo, are you sure you are {my_age}?"
```

```
How old are you?-20
```

EXAMPLE 2

```
def my_score(score):
   assert (score > 0 ),"Alo, are you okay?"
   return ((score / 50)*100)

print(my_score(25))
print(int(my_score(10)))
print(my_score(-5))
```

EXAMPLE 3

```python
def square(num):
    assert isinstance(num, (int, float)), "Input must be a number"
    return num * num

print(square(5))  # This works
print(square("5"))  # This raises an AssertionError
```

Pre-conditions

```python
def add_numbers(a, b):
    result = a + b
    # Ensure the result is greater than or equal to both numbers
    assert result >= a and result >= b, "Addition result is incorrect"
    return result

print(add_numbers(5, 10))  # Works fine
print(add_numbers(-1, -10))  # Also works fine
```

Post-conditions



WHEN TO USE ASSERTIONS?

- Debugging and testing during development
- To check preconditions/postconditions (e.g., function inputs/outputs)
- Internal sanity checks in algorithms

However, assertions should not be used to handle runtime errors or input validation in production code since they can be disabled using the -O flag. For runtime checks, raise appropriate exceptions.

"optimize"- used when running Python scripts to apply certain optimizations and disable assertions

HANDLING EXCEPTIONS

- An exception is an error that occurs during program execution. It interrupts the normal flow of the program. If not handled, Python stops the program and displays an error message (traceback).
- Exception handling: a process of responding to the occurrence of exceptions. Exception handling allows you to:
  - Prevent the program from crashing.
  - Control how errors are managed.
  - Provide meaningful feedback to users.
  - Continue program execution safely.
  - Etc.

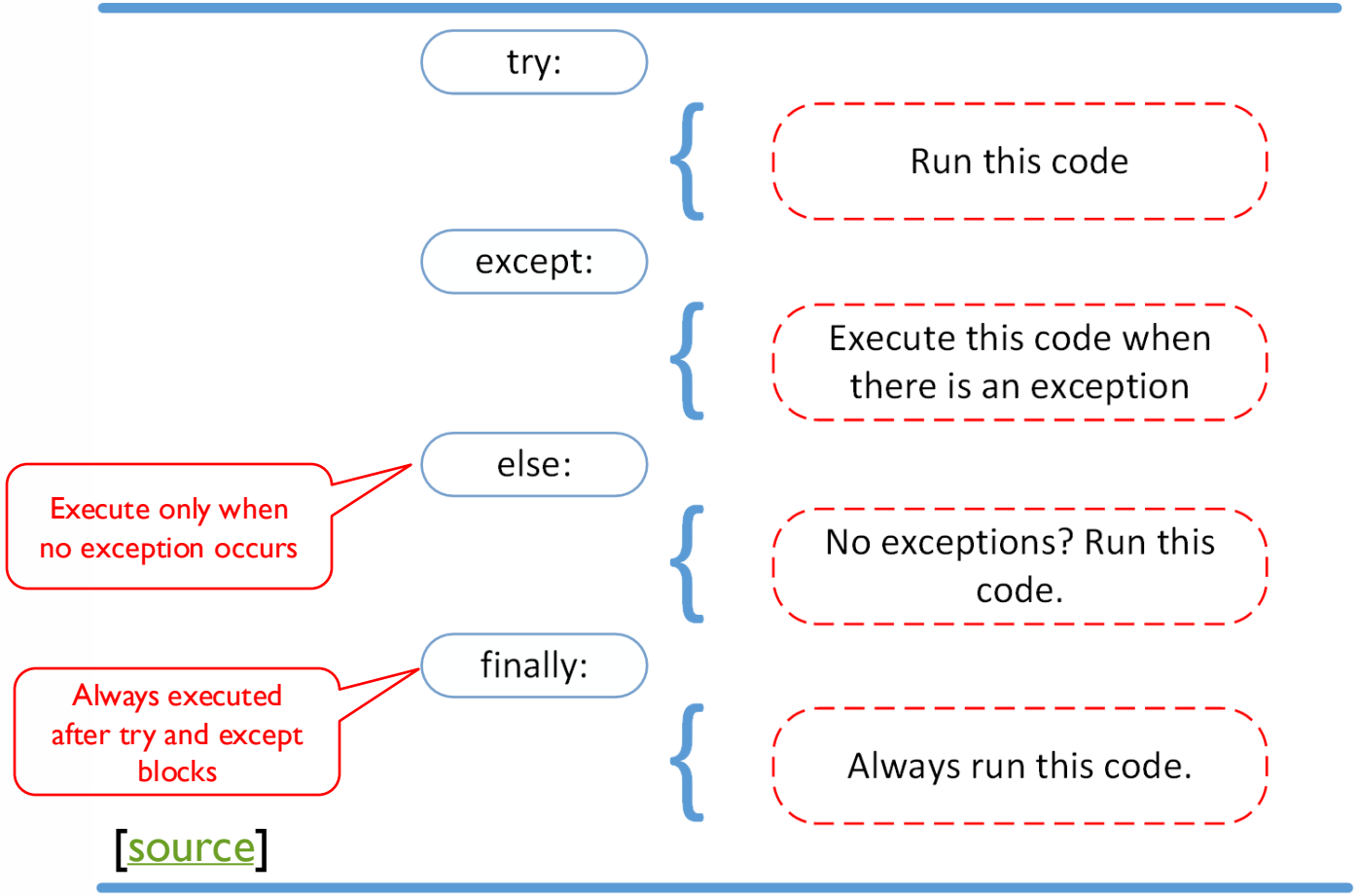
EXCEPTION HANDLING IN PYTHON

- `try`: Contains code that might throw an exception.

- `except`: Handles specific errors (e.g., ZeroDivisionError).

- `else`: Executes only if no exception occurred.

- `finally`: Always executes



PYTHON BUILT-IN EXCEPTIONS

Exception	Cause of Error
`AssertionError`	Raised when an `assert` statement fails.
`AttributeError`	Raised when attribute assignment or reference fails.
`EOFError`	Raised when the `input()` function hits end-of-file condition.
`FloatingPointError`	Raised when a floating point operation fails.
`GeneratorExit`	Raise when a generator's `close()` method is called.
`ImportError`	Raised when the imported module is not found.
`IndexError`	Raised when the index of a sequence is out of range.
`KeyError`	Raised when a key is not found in a dictionary.
`KeyboardInterrupt`	Raised when the user hits the interrupt key (`Ctrl+C` or `Delete`).
`MemoryError`	Raised when an operation runs out of memory.
`NameError`	Raised when a variable is not found in local or global scope.
`NotImplementedError`	Raised by abstract methods.
`OSError`	Raised when system operation causes system related error.
`OverflowError`	Raised when the result of an arithmetic operation is too large to be represented.
`ReferenceError`	Raised when a weak reference proxy is used to access a garbage collected referent.
`RuntimeError`	Raised when an error does not fall under any other category.
`StopIteration`	Raised by `next()` function to indicate that there is no further item to be returned by iterator.
`SyntaxError`	Raised by parser when syntax error is encountered.
`IndentationError`	Raised when there is incorrect indentation.
`TabError`	Raised when indentation consists of inconsistent tabs and spaces.
`SystemError`	Raised when interpreter detects internal error.
`SystemExit`	Raised by `sys.exit()` function.
`TypeError`	Raised when a function or operation is applied to an object of incorrect type.
`UnboundLocalError`	Raised when a reference is made to a local variable in a function or method, but no value has been bound to that variable.
`UnicodeError`	Raised when a Unicode-related encoding or decoding error occurs.
`UnicodeEncodeError`	Raised when a Unicode-related error occurs during encoding.
`UnicodeDecodeError`	Raised when a Unicode-related error occurs during decoding.
`UnicodeTranslateError`	Raised when a Unicode-related error occurs during translating.
`ValueError`	Raised when a function gets an argument of correct type but improper value.
`ZeroDivisionError`	Raised when the second operand of division or modulo operation is zero.

EXAMPLE 3

```
try:
    age = int(input("Please enter your age: "))
    print(f"Wow, you are {age} years old.")
except ValueError:
    print("Alo, are you kidding me?")
```

EXAMPLE 4

```
try:
    dividend = int(input("Please enter the dividend: "))
    divisor = int(input("Please enter the divisor: "))
    ans = dividend / divisor
    print(f"{dividend} / {divisor} = {ans}")
except(ValueError, ZeroDivisionError):
    print("Alo, something went wrong!")
```

EXAMPLE 5

```
try:
    dividend = int(input("Please enter the dividend: "))
    divisor = int(input("Please enter the divisor: "))
    ans = dividend / divisor
    print(f"{dividend} / {divisor} = {ans}")
except(ValueError, ZeroDivisionError):
    print("Alo, something went wrong!")
else:
    print("Everyting seems to be alright!")
```

Execute only when no exception occurs

```
Please enter the dividend: a
Alo, something went wrong!
```

EXAMPLE 6

```
try:
    dividend = int(input("Please enter the dividend: "))
    divisor = int(input("Please enter the divisor: "))
    ans = dividend / divisor
    print(f"{dividend} / {divisor} = {ans}")
except(ValueError, ZeroDivisionError):
    print("Alo, something went wrong!")
else:
    print("Everyting seems to be alright!")
finally:
    print("You are just awesome!")
```

Always executed after try and except blocks

```
Please enter the dividend: 2
Please enter the divisor: 8
2 / 8 = 0.25
Everyting seems to be alright!
You are just awesome!
```